Singapore Epidemic Simulator

Singapore Epidemic Simulator *(student proposal)*

> **Assessment:** ◆ Fit with adjustments · **Category:** Simulator · **Assessed:** 2026-07-13 · **Status:** awaiting instructor approval

Proposal (as submitted): A simulation of an epidemic in Singapore. The starting location can be any hospital or laboratory. A server runs the simulation in real time, where the spread is only caught after 5 days; citizens have a mobile-facing app that alerts them which zones to avoid and where it is safe to gather.

Category note: the deliverable IS a spatial disease simulation, so this is **Simulator** — the direct sibling of the category's Epidemic-spread example, extended with a Singapore geography and a citizen-facing alert client. The citizen-service angle makes **GovTech** the runner-up, but the scientific simulation engine (not the service workflow) is the heart of the project, so Simulator is the right home.

Adjustments required for approval:

- The spread must be a **load-bearing spatial C++ simulation** — Singapore modelled as zones (e.g. planning areas) with a per-tick metapopulation-SIR or agent-based transmission model and inter-zone mobility. It must NOT be a closed-form epidemic curve, a scripted animation, or a Firebase query; the real-time loop over the zone graph is the deliverable.
- **Simulated data only, unmistakably labelled.** The citizen app must be clearly a simulation / what-if exercise at all times — never presented as real outbreak data or official Singapore public-health guidance (a fictional outbreak alert mistaken for real would cause genuine harm). Reframe "where to gather" as *lower-risk zone guidance within the simulation*; drop any real-world instruction that marshals people to gather (sound outbreak guidance never directs crowds together).
- **T1 scope = local backend; the always-on real-time server is the Server-indicated T2 evolution.** In trimester 1 the C++ engine runs locally, the

citizen mobile app is a client, and Firebase distributes per-zone risk state and run results. Do not over-invest in distributed server infrastructure in T1 — the module's server platform is a second-trimester goal (Appendix A).

- **Keep the Simulator's scientific spine.** The "caught only after 5 days" detection lag must be a modelled, data-driven mechanic (a *hidden-infected* state vs. an *officially-detected* state, with the alerts firing off the lagging detected state). The project must retain a what-if dimension — comparing seed origins (which hospital/lab) and interventions across seeded runs — rather than collapsing into a live alert feed.

References:

- GLEAMviz — the Global Epidemic and Mobility Model (client-server, geo-mapped)
- NetLogo epiDEM Travel and Control
- HealthMap — global disease-alert map (citizen-facing, zone/dot alerts)

Core Tech Requirements

Legend: ✔ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✘ Not needed | » Second trimester

Core Tech Requirement	Singapore Epidemic Simulator
External Database (Firebase)	✔
Authentication »	●
Analytics »	✔
C++ Performant Service	✔
Real-time C++ Simulation Loop	✔
Application Communication	✔
Continuous Integration	✔
Data-Driven Architecture	✔
Front-end + Local Backend	✔
Custom Tools Development	✔
2D Graphical Output	✔
3D (strong students only)	✘
Networking System	●
Headless Batch Mode — run N simulations without rendering + statistical reports (new)	✔

How each requirement applies:

> *Platform fit:* **Native mobile + web browser on mobile (citizen app) + Native PC (ops/authoring)** — the citizen alert app is the mobile deliverable (reference device Pixel 8a/9a and/or mobile web); the operator/simulation view and scenario authoring run on the PC side.

Front end: Citizen app — Singapore zone map coloured by risk, "my-zone" status, alerts, lower-risk-zone guidance. Operator view — zone map with live SIR overlay, run controls (ImGui), SIR curves. **Languages:** C++, GLSL; HTML/JS/CSS (citizen app via WASM or mobile web).

Back end: Metapopulation/agent SIR engine with inter-zone mobility and a hidden-vs-detected (5-day lag) model, threshold→alert engine, headless campaign runner, Firebase for per-zone risk distribution and run results. **Languages:** C++.
Backend scope: Server-indicated (T2) — a real-time server pushing zone alerts to many citizen phones is inherently a shared-server design; T1 uses the local engine + Firebase as the stopgap.

- *Simulation loop:* per-tick transmission across the zone graph, SIR state transitions, inter-zone mobility, and the hidden→detected lag — the load-bearing core.
- *Data-driven:* disease parameters, zone/mobility graph, seed origin (hospital/lab), and intervention policies all as data. *Custom tool:* outbreak/scenario editor.
- *2D:* Singapore zone map coloured by risk plus live SIR curves. *Alerts:* per-zone threshold rules fire the citizen advisories (off the lagging detected state).
- *External DB / Analytics »:* per-zone risk state pushed to citizen clients; run results and scenario library in Firebase. *Auth »:* optional accounts. *Networking ●:* the real-time push is Firebase in T1, a dedicated server in T2.
- *Headless batch mode:* sweep seed origins and interventions across hundreds of seeded runs — "which origin/policy contains it fastest" is the scientific showcase result.

Suggested Tech Goals

The goals below are the **CS track**: G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Simulator pool in the CS Goals Pool (`../../Project 2 Technical adjustments/milestone-goals-pool-per-category.pdf`; G1–G3 ★ common, G4+ picks). The violet **Track goals** box below suggests the Design/Art/TEAM picks.

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic**
  Tech & dependencies
  - **Tech:** SDL2 window/input, fixed-timestep loop applying per-tick inter-zone transmission, SIR transitions and the hidden→detected lag over the Singapore zone graph; CMake, Git.
  - **Prerequisites:** None — foundation of this project.
  - **Feeds into:** M1 G4 (renders its world), M1 G14 (watches its detected counts); every other goal.
- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**
  Tech & dependencies
  - **Tech:** Disease model (transmission probability, recovery time, incubation), the hidden-infected vs. officially-detected states (the 5-day lag), zone attributes and seed origin defined in JSON (nlohmann/json) with hot-reload.
  - **Prerequisites:** G1 (shell & loop).
  - **Feeds into:** M1 G15 (zone/mobility graph as data), M1 G14 (alert thresholds), M2 G2 (editor edits this model).
- **G3 ★ Debugging, Profiling, Stress Test & CI**
  Tech & dependencies
  - **Tech:** ImGui overlays of per-zone SIR counts and hidden-vs-detected gap, Tracy profiling at full-island population, Catch2 tests of transition and lag logic, GitHub Actions Windows + Linux builds.
  - **Prerequisites:** G1 (shell & loop).
  - **Feeds into:** M2 G3 (CI advanced); M2 G16 (stress hooks reused by the campaign runner).

- **G4 2D Visualization of the Simulated World** — the Singapore zone map coloured by infection/risk plus live SIR curves.
  Tech & dependencies
  - **Tech:** SDL2/OpenGL rendering of the Singapore zone map shaded by per-zone SIR/risk state, with live SIR curves and FreeType labels.
  - **Prerequisites:** G1 (loop, per-zone state) and G15 (zone geometry to draw).
  - **Feeds into:** M1 G14 (alerts overlay it); M2 G4 (advanced heatmap) and M2 G14 (the citizen app reuses this renderer).
- **G14 Alert & Feedback System** — per-zone risk thresholds fire advisories off the lagging *detected* state.
  Tech & dependencies
  - **Tech:** Threshold rules (per-zone detected-case / capacity breach) firing visual alarms via SDL2/ImGui and audio cues via OpenAL, driven by the detected (not the true) state so the 5-day lag is felt.
  - **Prerequisites:** G1 (live detected counts); overlays on M1 G4.
  - **Feeds into:** M2 G14 (the citizen app surfaces these alerts on the phone); alert events logged for M2 G15 reports.
- **G15 Map / Environment System** — Singapore as a zone + mobility graph.
  Tech & dependencies
  - **Tech:** Singapore planning areas as a JSON zone graph (adjacency + commuting weights), pan/zoom map camera, spatial lookup; seed origin (hospital/lab) placed on the graph.
  - **Prerequisites:** G2 (data model).
  - **Feeds into:** M1 G1 (transmission spreads across it), M1 G4 (drawn as the map), M2 G2 (editor authors zones/seeds).

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**
  Tech & dependencies
  - **Tech:** Tracy profiling of full-island populations, memory pools, std::thread; Emscripten/WASM build of the citizen app validated on the mobile target (Pixel 8a/9a and/or mobile web).
  - **Prerequisites:** All M1 goals (hardens the M1 codebase).
  - **Feeds into:** M2 G14 (mobile citizen app) and M2 G16 (fast headless campaigns).

- **G2 ★ Content Editor — Core** — the outbreak/scenario editor.
  Tech & dependencies
    - **Tech:** ImGui editor to author zones and mobility weights, place the seed origin (hospital/lab), and set disease/detection-lag/intervention parameters, with JSON round-trip and undo/redo.
    - **Libraries/Software:** Dear ImGui, nlohmann/json, project's SDL2/OpenGL zone-map renderer (live preview)
    - **Prerequisites:** M1 G2 (data model) and M1 G15 (zone/mobility format it edits).
    - **Feeds into:** Scenarios queued by M2 G16 and shared through Firebase.
- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**
  Tech & dependencies
    - **Tech:** CMake presets, clang-format + clang-tidy, packaging of the native ops app and the Emscripten citizen-app build via GitHub Actions matrix.
    - **Prerequisites:** M1 G3 (CI foundation).
    - **Feeds into:** Release/showcase builds of every goal.
- **G4 Visualization — Advanced** — animated zone-risk heatmap with the detected-vs-actual lag made visible.
  Tech & dependencies
    - **Tech:** GLSL risk heatmap over the Singapore map, time scrubbing of an outbreak, and a detected-vs-actual overlay that dramatises the 5-day blind window.
    - **Prerequisites:** M1 G4 (base map/curves) and M1 G15; M2 G1 (frame budget).
    - **Feeds into:** The showcase; the visual layer the citizen app (M2 G14) draws from.
- **G14 Mobile / Web Viewer Port** — the citizen alert app on the phone.
  Tech & dependencies
    - **Tech:** Emscripten/WASM (or mobile-web) citizen client — zone-risk map, "my-zone" status, alerts and lower-risk-zone guidance — subscribing to per-zone risk state from Firebase; touch input, mobile frame budget; a persistent "SIMULATION — not real health advice" label.
    - **Prerequisites:** M2 G1 (optimised, mobile-validated build) and M1 G14 (the alert logic it surfaces).

    - **Feeds into:** The mobile showcase deliverable — the citizen half of the product.
- **G16 Multi-Scenario Campaign Runner** — sweep seed origins and interventions across seeded runs.
  Tech & dependencies
    - **Tech:** CLI-driven std::thread run farm queueing hundreds of seeded, headless runs across seed origins (which hospital/lab) and interventions (distancing, zone lockdown, travel limits), CSV/JSON output.
    - **Prerequisites:** M1 G1 (render-decoupled core), M1 G3 (stress hooks), M1 G15 (zone graph) and M2 G2 (scenarios).
    - **Feeds into:** The scientific showcase result — which origin/policy contains the outbreak fastest; data behind M2 G4 overlays.

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Recommended composition:** 6 CS + 2 Design + 0 Art — two front-ends (operator simulation view + citizen alert app) plus a Singapore zone map give a substantial UX/service-design surface, so 2 designers are justified (one on the simulation/ops UX, one on the citizen app + map legibility). Art surface is low (map styling, zone iconography), so 0 artists with the Design conditionals covering it; 1 Art is optional if the team wants a polished animated map.
- **Design M1:**
    - D1 ★ Features Design Document — the outbreak model and its two audiences (operators reading the spread, citizens reading their zone)
    - D2 Prototype — configure outbreak → run → citizen sees a zone alert, clickable end to end
    - D7 Parameter Model design — infection, mobility and the 5-day detection-lag parameters
- **Design M2:**
    - D1 ★ Features Design Iteration & Showcase Readiness
    - D6 Polished experiment — one outbreak scenario end to end incl. citizen alerts
    - D8 Legibility pass — zone-risk map readability (colour-blind-safe risk scale)

- **UI/UX M1** (graded by the UI/UX instructor):
  - U1 ★ UI Standards & Wireframe Baseline — standards for BOTH the ops sim view and the citizen app
  - U2 UX Flows & Wireframes — the citizen app screens and the operator screens
- **UI/UX M2:**
  - U1 ★ Functional UI & Usability — screens: (ops) scenario select · run view with pause/speed · report; (citizen) zone map · my-zone risk · alerts · lower-risk-zone guidance; both navigable unaided
  - U2 ★ UX Validation & Final Polished UI — validated with real users; polished citizen + ops UI on the mobile target
  - U5 Mobile UX Adaptation & Ergonomics — the citizen app on the phone (thumb-zone layout, glanceable my-zone status)
- **Art:** 0 artists — the Design conditionals (M1 D11 Art POC / M2 D11 Art Implementation Quality) cover the map styling and zone iconography; 1 Art optional for a polished animated Singapore map.
- **TEAM M1:**
  - TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present and working in the prototype, suitable and correctly triggered for this project.
  - TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add or replace audio without recompiling) and reliable multi-channel playback.
- **TEAM M2:**
  - TA1 ★ Overall Audio Experience — Simulator floor: BGM optional; ≥5 event cues (outbreak start, threshold breach, alert issued, zone cleared, run complete), each with a visual pair for the citizen app (accessibility).
  - TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel; advanced where the project calls for it).
  - TP1 ★ Final Presentation — 7 min in front of class + instructors: live showcase of the outbreak/scenario editor, prototype demo on the mobile citizen app, team post-mortem (wk14).

Generated by the Project Proposal Assessor skill · grounding: live folder · references verified 2026-07-13.